

Reviewer Pack — Minimal Rank of Quasi-Polynomial L-Functions over Boolean Ci...

Entry 97c6fcaa7ee0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 12:22:50 UTC

Minimal Rank of Quasi-Polynomial L-Functions over Boolean Circuit Satisfiability

Entry ID: 97c6fcaa7ee0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 12:22:50 UTC

1. Conjecture

Field A (mathematical branch): Analytic Number Theory (Quasi-Polynomial L-Functions) **Field B** (complexity object): Complexity Theory: Boolean Circuit Satisfiability

Statement:

[‘For every explicit boolean function f in P , the minimal rank of the associated quasi-polynomial L-function R_f is $\Theta(\log n)$, where n is the number of variables.’, ‘Equivalently, for any circuit C with depth d and size s that computes f , there exists a quasi-polynomial L-function R_C associated with C such that the rank of R_C is $\Theta(d \log^2 s)$.’, ‘If a boolean function has an L-function with rank less than $\Theta(\log n)$, then it can be computed by a circuit with depth less than $\log n$.’]

Rationale (proposer’s reasoning):

[‘Quasi-polynomial L-functions provide a bridge between number theory and complexity theory, particularly in the study of computational hardness. By examining the rank of these functions, we may expose hidden structures that are not evident through standard complexity-theoretic measures.’, ‘The conjecture suggests that certain properties of boolean functions may be encoded in their associated L-functions, potentially leading to new algorithms or bounds for circuit satisfiability.’]

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9cea70569f1e3ab5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of a quasi-polynomial L-function associated with an explicit boolean function is within $\Theta(\log n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quasi-polynomial L-functions" AND "Boolean Circuit Satisfiability"
- "minimal rank" AND "L-function" AND (circuit OR boolean function) - "depth d and size s"
AND " $\theta(d \log^2 s)$ " AND L-function

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_l_function(f):
        n = int(math.log2(len(f)))
        if f == [0]*len(f) or f == [1]*len(f):
            return 1
        rank = 0
        for i in range(n):
            sub_f = []
            for j in range(2**n):
                if (j >> i) & 1:
                    sub_f.append(f[j])
            rank += compute_l_function(sub_f)
        return rank

    def is_prime(num):
        if num <= 1:
            return False
        if num == 2:
            return True
        if num % 2 == 0:
            return False
        for i in range(3, int(math.sqrt(num)) + 1, 2):
```

```

        if num % i == 0:
            return False
        return True

def generate_primes(k):
    primes = []
    candidate = 2
    while len(primes) < k:
        if is_prime(candidate):
            primes.append(candidate)
            candidate += 1
    return primes

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_boolean_function(n)
    rank = compute_l_function(f)
    results.append({"n": n, "rank": rank})

mean_rank = sum(result["rank"] for result in results) / len(results)
std_deviation = math.sqrt(sum((result["rank"] - mean_rank) ** 2 for result in results) / len(results))

conjecture_holds = all(mean_rank >= n * math.log(n, 2) and mean_rank <= n * math.log(n, 2) + 1 for n in n_values)
counterexample = "" if conjecture_holds else "rank_outside_bound"

return {
    "metric_name": "mean_rank",
    "metric_value": mean_rank,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] or generate_primes(30)

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    std_deviation = math.sqrt(sum((result["metric_value"] - mean_rank) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_deviation} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_deviation} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"rank_outside_bound\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm or refute the conjecture based on the results. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13596
2	propose	ollama_remote	glm4:latest	0	0	10594
3	propose	ollama_remote	glm4:latest	0	0	10576
4	preregistration	ollama_remote	glm4:latest	0	0	5410
5	novelty	ollama_remote	glm4:latest	0	0	4897
6	novelty	ollama_remote	glm4:latest	0	0	5718
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27696
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8377
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7007
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9333
11	judge	ollama_remote	glm4:latest	0	0	27215

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 130419 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/97c6fcaa7ee0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/97c6fcaa7ee0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/97c6fcaa7ee0.tar.gz (if generated)